

-  Home
-  Library
-  Profile
-  Stories
-  Stats

 Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



InsiderFinance Wire · [Follow publication](#)

How I Built a Three-Agent AI Trader That Argues With Itself Before Placing a Trade

15 min read · Jul 24, 2025



Gaille Amolong

Follow

 Listen

 Share

 More

I prototyped an open-source “Layered-Memory Trader” (LMT) that lets three specialised agents; short-term, swing, and macro — debate a trade, weigh their confidence, and act only when they’re in solid agreement. No real capital is at risk in this article; results are limited to back-tests, so treat everything here as research, not investment advice. Inspired by Li et al.’s 2023 “[TradingGPT](#)” paper ([CC BY 4.0](#)). If you’d like to dig deeper or support the project, check this [repository](#) and my [LinkedIn](#) is one click away.

Picture the 2 a.m. split-brain moment every trader dreads. You’re bleary-eyed in front of the charts, RSI screams oversold, the daily trend still slopes down, your gut whispers bounce, while your risk rules mutter “stay flat.” That tug-of-war, short-term noise versus long-term structure, hard

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



metrics versus intuition, made me pause when I read the TradingGPT paper and thought, *what if software could host that same internal debate, remember each argument, and act only when the voices finally agree?* Chasing that idea down the rabbit hole led me to create a version from that paper — LMT, the Layered-Memory Trader.

A Team of Tireless Analysts in Software

Instead of forcing you to choose between day-trading instincts and swing-trading logic, LMT embraces the chaos. It spins up three “analyst” agents, each with its own specialty and memory buffer, then lets them hash it out:

1. **Short-Term Agent** — Lives on minute-level momentum, perfect for tight entries and exits.
2. **Swing-Term Agent** — Tracks multi-day structure to filter noise and reinforce — or veto — fast signals.
3. **Macro Agent** — Scans weeks-to-months of price action and piping in news sentiment for the bigger picture.

Only when a confidence-weighted vote clears a threshold does LMT commit capital (well, simulated capital for now).

What You'll See Next

Here's the roadmap for the rest of the article. First, I'll show how raw price feeds are transformed into indicator-rich data frames; next, you'll see how LMT keeps three lean yet informative memory buffers, one each for short-term, swing, and macro context. From there, we'll dive into the debate engine, where the agents cast weighted votes that determine whether to stay on the sidelines or pull the trigger. Finally, I'll

Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data ...
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

unpack the back-test: equity curve, Sharpe ratio, drawdowns, and the missteps that surfaced. Whether you're a quant hunting new architectures or a discretionary trader curious about AI, LMT offers a glimpse of where multi-agent thinking can push trading systems. So let's dive in.

1. System Overview

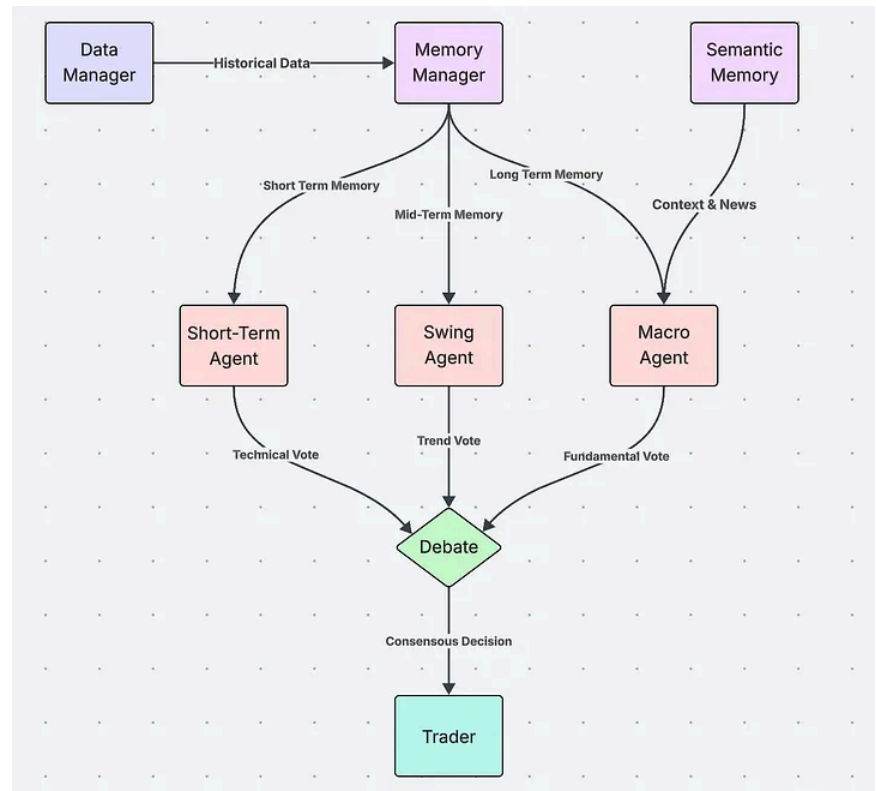


Figure 1. Layered-Memory Trader architecture

LMT starts with a Data Manager that pulls raw candles and sentiment, then hands everything to a Memory Manager that slices the stream into three rolling buffers: short, swing, and macro.

- **Short-Term Agent** grabs the < 60-minute buffer, hunts momentum, and casts a *technical* vote.
- **Swing Agent** leans on multi-day data for a *trend* vote.
- **Macro Agent** reads weeks-to-months of price plus semantic-news vectors for a *fundamental* vote.

Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •

Those three ballots flow into the Debate Engine, which runs a confidence-weighted election. Only if the net conviction clears a configurable bar does the Trader module fire an order (simulated in our back-tests).

1.1. Data Ingestion and Market Intelligence Gathering

Everything the three agents know about the market comes from one place, which is the DataManager. Its job is simple but critical. First, it loads historical candles (or live feeds later) for the tickers in scope. Second, it partitions the date range into train and test sets for back-tests. Third, it exposes a function *get_data_for_ticker()*, which is a helper that each agent can call on demand. Below is the full class so you can paste it straight into your project.

```
from data.price_fetcher import fetch_price_data
from data.sentiment_fetcher import fetch_sentimen
import pandas as pd
import time
import yaml

class DataManager:
    def __init__(self, config, backtest_mode=None):
        self.config = config
        self.backtest_mode = backtest_mode
        self.all_historical_data = pd.DataFrame()
        self.tickers = []

        if self.backtest_mode:
            self._load_backtest_data()

    def _load_backtest_data(self):
        """Loads and prepares historical data for
        # Try to load data with news first, fallb
        news_data_path = self.config['backtest'].
        data_path = self.config['backtest']['full

        try:
            print(f"Attempting to load data with
            self.all_historical_data = pd.read_cs
            news_data_path,
```

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



```

        parse_dates=['time']
    )
    print(f"Successfully loaded data with
except FileNotFoundError:
    print(f"News data not found at {news_
self.all_historical_data = pd.read_cs
    data_path,
    parse_dates=['time']
    )
    # Add placeholder news column if not
    if 'news_summary' not in self.all_his
    self.all_historical_data['news_su

self.tickers = self.all_historical_data['

if self.backtest_mode == 'train':
    start_date = pd.to_datetime(self.conf
    end_date = pd.to_datetime(self.config
elif self.backtest_mode == 'test':
    start_date = pd.to_datetime(self.conf
    end_date = pd.to_datetime(self.config
else:
    raise ValueError("Invalid backtest mo

self.all_historical_data = self.all_histo
    (self.all_historical_data['time'] >=
    (self.all_historical_data['time'] <=
    ]
self.backtest_iterator = self.all_histori
print(f"Historical data for {self.backtes

# Check if news data is available
if 'news_summary' in self.all_historical_
    news_count = self.all_historical_data
    print(f"News data available: {news_co
else:
    print("No news data available in the

def get_data_for_ticker(self, ticker):
    """Returns the historical data for a spec
    if self.backtest_mode:
        try:
            return self.backtest_iterator.get
        except KeyError:
            return pd.DataFrame()
    else:
        # Live mode would fetch data for a sp
        return pd.DataFrame()

if __name__ == '__main__':
    with open('config.yaml', 'r') as f:
        config = yaml.safe_load(f)

```

Medium

[Home](#)[Library](#)[Profile](#)[Stories](#)[Stats](#)[Following](#)[Kiran Maan](#)[ODSC - Open Data ...](#)[Data Engineer Thin...](#)[Data Hackers](#)[tecnologia no Grup...](#)[DataDrivenInvestor](#)[Practicus AI](#)[The Startup](#)[Entrepreneurship ...](#)[Data Science Colle...](#)

```
data_manager = DataManager(config, backtest_m
```

```
aapl_data = data_manager.get_data_for_ticker(  
print("AAPL Data:")  
print(aapl_data.head())
```

Once the raw candles are in memory, LMT augments them with a handful of momentum and volatility signals that every agent understands:

```
# Technical indicator enhancement  
ticker_data['rsi'] = calculate_rsi(ticker_data)  
ticker_data['macd'], ticker_data['macd_signal'] =  
ticker_data['upper_band'], ticker_data['lower_ban
```

This enriched frame is what flows into the Memory Manager next, ensuring each agent sees not just price but also momentum context before casting its vote.

1.2. The Layered Memory

The heart of LMT lies in the MemoryManager class, which is a component that mimics how human traders naturally organize and recall market information. Human traders don't treat every tick the same; they keep an intraday pulse, a swing view, and a long-term thesis in separate mental buckets. LMT copies that habit with a MemoryManager that maintains three rolling buffers, plus a reflection log for post-trade notes.

```
import pandas as pd  
import yaml  
  
class MemoryManager:  
    def __init__(self, horizons: dict):  
        self.horizons = horizons
```

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



```

self.short_term_memory = pd.DataFrame()
self.mid_term_memory = pd.DataFrame()
self.long_term_memory = pd.DataFrame()
self.reflection_memory = pd.DataFrame(col

def update_memory(self, new_data: pd.DataFram
"""
Updates all memory layers with new data a

:param new_data: A DataFrame containing t
"""
if new_data.empty:
    return

# Update short-term memory
self.short_term_memory = pd.concat([self.
if len(self.short_term_memory) > self.hor
    self.short_term_memory = self.short_t

# Update mid-term memory
self.mid_term_memory = pd.concat([self.mi
if len(self.mid_term_memory) > self.horiz
    self.mid_term_memory = self.mid_term_

# Update long-term memory
self.long_term_memory = pd.concat([self.l

def add_reflection(self, timestamp, decision,
new_reflection = pd.DataFrame({
    'timestamp': [timestamp],
    'decision': [decision],
    'confidence': [confidence],
    'outcome': [outcome],
    'reflection': [reflection]
})
self.reflection_memory = pd.concat([self.

def get_memory_snapshot(self) -> dict:
"""
Returns a dictionary containing the curre
"""
return {
    'short_term': self.short_term_memory,
    'mid_term': self.mid_term_memory,
    'long_term': self.long_term_memory,
    'reflections': self.reflection_memory
}

```

How the three layers work in practice:

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



- **Short-term memory** holds the freshest ticks — price bursts, volume spikes, momentum shifts — so the intraday agent can fire quickly without being drowned by old data.
- **Mid-term memory** keeps several days to a few weeks, letting the swing agent spot trend continuations, pullbacks, or emerging ranges that a 200-bar window would miss.
- **Long-term memory** stretches back months, giving the macro agent enough context to judge whether the current action is a blip or a regime change.

LMT avoids the classic pitfall of information overload while still preserving the context each agent truly needs, that is, if it's capping each buffer to its horizon. In other words, every analyst sees exactly the slice of market history that matches its job description; no more, no less.

1.3. The Specialized Agent Team: Different Minds, Common Goal

Once the market stream is bucketed into three memories, LMT assigns a dedicated analyst to each horizon. Every agent inherits from the same BaseAgent interface; so they share logging, risk limits, and config parsing, but each brings its own prompt template and voting logic.

```
from abc import ABC, abstractmethod
import pandas as pd
from memory.semantic_memory import SemanticMemory

class BaseAgent(ABC):
    """
    Abstract base class for all trading agents.
    """
    def __init__(self, name: str, config: dict, s
        """
        Initializes the agent.
```

Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •

```
:param name: The name of the agent (e.g.,
:param config: A configuration dictionary
:param semantic_memory: An instance of Se
"""
```

```
self.name = name
self.config = config
self.semantic_memory = semantic_memory
```

Short-Term Agent: This agent thinks like a scalper. It scans the most recent few hundred bars for breakouts and momentum shifts, then asks a lightweight LLM prompt for a binary buy / sell / hold with confidence. High speed, high noise, high conviction.

```
import pandas as pd
from agents.base_agent import BaseAgent
from memory.semantic_memory import SemanticMemory
import google.generativeai as genai
import os
import re

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

class ShortTermAgent(BaseAgent):
    """
    Agent focusing on short-term data to make tra
    """
    def __init__(self, name: str, config: dict, s
        super().__init__(name, config, semantic_m
        self.model = genai.GenerativeModel('gemin

    def vote(self, memory_snapshot: dict) -> tuple
        """
        Analyzes short-term memory using an LLM t
        """
        short_term_data = memory_snapshot.get('sh
        if short_term_data is None or short_term_
            return 'HOLD', 0.5

        ticker = short_term_data['ticker'].iloc[-

        # Prepare the Prompt
        prompt = f"You are a short-term momentum
        prompt += f"Short-Term Price & Indicator
        prompt += short_term_data[['close', 'rsi'
```

Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •

```
# Get LLM Response
```

```
try:
```

```
    response = self.model.generate_content(prompt)
    # Parse the Response
    vote_match = re.search(r"VOTE:\s*(BUY|SELL|HOLD)", response)
    confidence_match = re.search(r"CONFIDENCE:\s*(\d+)", response)
```

```
    if vote_match and confidence_match:
        vote = vote_match.group(1).upper()
        confidence = float(confidence_match.group(1))
        print(f"ShortTermAgent LLM Vote: {vote} Confidence: {confidence}")
        return vote, confidence
```

```
    else:
        print(f"ShortTermAgent: Could not parse LLM response")
        return 'HOLD', 0.5
```

```
except Exception as e:
```

```
    print(f"An error occurred while calling ShortTermAgent: {e}")
    return 'HOLD', 0.5
```

Mid-Term Agent: This agent lives in the middle. It pulls the mid-term buffer, days to weeks, calculates moving-average crosses, RSI divergences, and checks if the short-term call fits the broader pattern. Its vote often moderates the scalper's enthusiasm.

```
import pandas as pd
from agents.base_agent import BaseAgent
from memory.semantic_memory import SemanticMemory
import google.generativeai as genai
import os
import re
```

```
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
```

```
class MidTermAgent(BaseAgent):
```

```
    """
```

```
    Agent focusing on mid-term data to make trading decisions
    """
```

```
    def __init__(self, name: str, config: dict, semantic_memory: SemanticMemory):
        super().__init__(name, config, semantic_memory)
        self.model = genai.GenerativeModel('gemini-1.5-pro')
```

```
    def vote(self, memory_snapshot: dict) -> tuple[str, float]:
        """
```

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



Analyzes mid-term memory using an LLM to
"""

```
mid_term_data = memory_snapshot.get('mid_
if mid_term_data is None or mid_term_data
    return 'HOLD', 0.5
```

```
ticker = mid_term_data['ticker'].iloc[-1]
```

```
# Prepare the Prompt
```

```
prompt = f"You are a mid-term trend analy
```

```
# Add mid-term price trend with moving av
```

```
prompt += f"Mid-Term Price & Indicator Da
```

```
prompt += mid_term_data[['close', 'rsi'],
```

```
prompt += "5-day Moving Average:\n"
```

```
prompt += mid_term_data['close'].rolling(
```

```
prompt += "20-day Moving Average:\n"
```

```
prompt += mid_term_data['close'].rolling(
```

```
# Get LLM Response
```

```
try:
```

```
    response = self.model.generate_conten
```

```
# Parse the Response
```

```
vote_match = re.search(r"VOTE:\s*(BUY
```

```
confidence_match = re.search(r"CONFID
```

```
if vote_match and confidence_match:
```

```
    vote = vote_match.group(1).upper(
```

```
    confidence = float(confidence_mat
```

```
    print(f"MidTermAgent LLM Vote: {v
```

```
    return vote, confidence
```

```
else:
```

```
    print(f"MidTermAgent: Could not p
```

```
    return 'HOLD', 0.5
```

```
except Exception as e:
```

```
    print(f"An error occurred while calli
```

```
    return 'HOLD', 0.5
```

Macro (Long-Term) Agent: This agent is the strategist. It reads months of price action plus a semantic search over recent news, Fed statements, or earnings chatter stored in the SemanticMemory class. If the macro view screams “risk-off,” it can overrule the other two by delivering a strong, opposite-direction vote.

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



```
import pandas as pd
from agents.base_agent import BaseAgent
from memory.semantic_memory import SemanticMemory
import google.generativeai as genai
import os
import re
```

```
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
```

```
class LongTermAgent(BaseAgent):
    """
    Agent focusing on long-term data and macroeco
    """
    def __init__(self, name: str, config: dict, s
        super().__init__(name, config, semantic_m
        self.model = genai.GenerativeModel('gemini

    def vote(self, memory_snapshot: dict) -> tuple
        """
        Analyzes long-term memory and semantic co
        """
        long_term_data = memory_snapshot.get('long
        if long_term_data is None or long_term_da
            return 'HOLD', 0.5

        ticker = long_term_data['ticker'].iloc[-1

        # Prepare the Prompt
        prompt = f"You are a long-term trading an

        # Add long-term price trend
        prompt += f"Long-Term Price & Indicator D
        prompt += long_term_data[['close', 'rsi',

        # Add semantic memory context
        try:
            semantic_results = self.semantic_memo
            if semantic_results:
                prompt += "Recent News & Reflecti
                for result in semantic_results:
                    prompt += f"- {result['text']}
        except (IndexError, ValueError):
            # Not enough memories to search or ot
            prompt += "No significant news or ref

        # Get LLM Response
        try:
            response = self.model.generate_conten
            # Parse the Response
            vote_match = re.search(r"VOTE:\s*(BUY
            confidence_match = re.search(r"CONFID
```

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



```

if vote_match and confidence_match:
    vote = vote_match.group(1).upper()
    confidence = float(confidence_mat
    print(f"LongTermAgent LLM Vote: {
    return vote, confidence
else:
    print(f"LongTermAgent: Could not
    return 'HOLD', 0.5

```

```

except Exception as e:
    print(f"An error occurred while calli
    return 'HOLD', 0.5

```

Because all three agents return a (vote, confidence) pair, the Debate Engine can treat them symmetrically, weighting each by conviction, spotting deadlocks, and opting to stand aside when consensus is weak. That diversity of thought, not raw model size, is what gives LMT its edge.

1.4. Debate: turning three opinions into one trade

After each agent casts a (vote, confidence) pair, LMT runs a confidence-weighted election rather than a simple majority. The logic mirrors an institutional trading desk: louder voices only matter if they're also more certain.

```

from typing import List, Tuple
from agents.base_agent import BaseAgent

class Debate:
    """
    Orchestrates a debate among trading agents to
    """
    def __init__(self, agents: List[BaseAgent]):
        self.agents = agents

    def run(self, memory_snapshot: dict) -> Tuple
        """
        Gathers votes from all agents and determi
        """

```

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



```

votes = []
for agent in self.agents:
    decision, confidence = agent.vote(mem
    votes.append({'agent': agent.name, 'd

    final_decision, final_confidence = self._

    return final_decision, final_confidence,

def _resolve_votes_with_weighting(self, votes
    """
    Resolves the collected votes into a singl
    """
    if not votes:
        return 'HOLD', 0.5

    buy_strength = sum(v['confidence'] for v
    sell_strength = sum(v['confidence'] for v

    # Normalize by the sum of all confidences
    total_confidence = sum(v['confidence'] fo
    if total_confidence == 0:
        return 'HOLD', 0.0

    # The final confidence is the difference
    net_strength = buy_strength - sell_streng
    final_confidence = abs(net_strength) / to

    if net_strength > 0:
        return 'BUY', final_confidence
    elif net_strength < 0:
        return 'SELL', final_confidence
    else:
        return 'HOLD', 1.0 - final_confidence

```

For instance, imagine the short-term agent comes in hot with BUY (0.9), the swing agent counters with SELL (0.4), and the macro agent opts for HOLD (0.2).

When the Debate engine sums the weighted opinions, buy strength is 0.9 and sell strength is 0.4, giving a net of 0.5. Dividing that by the total confidence (0.9 + 0.4 = 1.3) yields an overall confidence of roughly 0.56, so the system's verdict is BUY at 56 % conviction.



Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data Science
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

Crucially, if that 0.56 score falls below the `mean_confidence_to_act` threshold set in the config, LMT simply stands aside; no trade, no churn. In this way, the debate layer functions as built-in risk control, throttling impulsive signals and ensuring capital only moves when the collective conviction is truly solid.

1.5. Backtesting Framework and Performance Analysis

LMT only earns its keep if it survives a cold, out-of-sample back-test. I took the 2023-24 price history, carved out clean train and test windows, and ran each agent in “paper-broker” mode with `trader.py` and `evaluate.py`, then rolled up the metrics you see below.

Decision distribution

Trade Decision Distribution (BUY/SELL/HOLD)

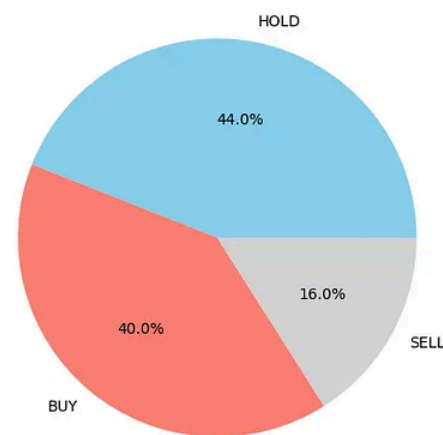


Figure 2: Trade Decision Distribution

Figure 2 shows the proportion of buy / sell / hold calls. Roughly 40 % buys, 16 % sells, and 44 % holds tells us two things: the system isn’t gun-shy, but it also parks capital when conviction is thin. Those HOLD slices

Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data ...
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

are a direct result of the debate layer filtering out low-edge noise.

The Agent vs. simple buy-and-hold

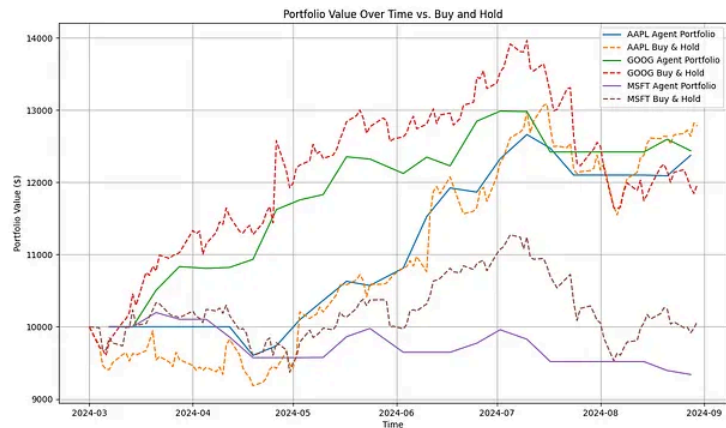


Figure 3: Portfolio Value Over Time Using the AI Agent vs Buy and Hold Strategy

Figure 3 plots LMT's equity curve against a passive benchmark. The multi-agent stack side-stepped the sharp May-June dip and re-deployed quickly when momentum returned, a capital-preservation reflex you'll never see in a static long-only portfolio.

Risk-adjusted scorecard

LMT's evaluation generates comprehensive performance analytics that go beyond simple return calculations:

```
Backtest finished.

Trade log saved to documentation/results/trade_log.csv

Backtest Evaluation Results:
Total Trades Executed: 42
Decision distribution chart saved to documentation/results/decision_distribution_pie_chart.png
Portfolio performance graph saved to documentation/results/portfolio_performance.png

AAPL Advanced Metrics:
Final Portfolio Value: $12,374.20
Sharpe Ratio: 2.65
Max Drawdown: -4.51%

GOOG Advanced Metrics:
Final Portfolio Value: $12,438.32
Sharpe Ratio: 2.64
Max Drawdown: -4.36%

MSFT Advanced Metrics:
Final Portfolio Value: $9,339.20
Sharpe Ratio: -1.22
Max Drawdown: -8.41%

Summary report saved to documentation/results/summary_report.md
```

Figure 4: Performance Analytics

Medium

 Home Library Profile Stories Stats Following Kiran Maan • ODSC - Open Data ... • Data Engineer Thin... • Data Hackers • tecnologia no Grup... • DataDrivenInvestor • Practicus AI • The Startup • Entrepreneurship ... • Data Science Colle... •

Figure 4 shows the headline numbers: Sharpe ratios of roughly 2.6 on AAPL and GOOG, with max draw-downs near -4 % (-8 % on MSFT after a rough earnings print). A Sharpe north of 2 means the returns more than paid for the volatility, and single-digit draw-downs keep both the psychology and any future position-sizer sane.

Reality check. Data quality drives everything here. For this demo I leaned on free end-of-day feeds for AAPL, GOOG, and MSFT and patched a few missing bars with simple interpolation, no premium tick data, no institutional news feed, and zero live fills. That's fine for proving out the architecture, but you should expect very different numbers once you swap in higher-resolution data, a broader universe, and real execution costs. Treat the curves above as a feasibility signal, not a performance promise.

Even with those caveats, the results hint that layered memory plus a confidence-weighted debate can do real work, not just curve-fit last year's tape.

Real-World Implementation Considerations

- **Friction costs.** Live trading means commissions, bid-ask spread, price slippage, and occasional market impact. In thin names, these can dwarf an edge. Any production roll-out needs a broker API wrapper that simulates all four costs during walk-forward tests.
- **Regime shifts.** A model trained on low-vol 2021 tape will struggle in a 2022 bear or a 2020 melt-up. Periodic re-training plus a “health check” that throttles position size when live Sharpe decays can keep the system aligned with the current regime.

Medium



Home



Library



Profile



Stories



Stats



Following



Kiran Maan



ODSC - Open Data ...



Data Engineer Thin...



Data Hackers



tecnologia no Grup...



DataDrivenInvestor



Practicus AI



The Startup



Entrepreneurship ...



Data Science Colle...



- **Latency vs. depth.** Three LLM prompts per bar is fine on hourly candles; it is suicide on a 50 ms crypto feed. In lower-latency venues, you'll want distilled, fast tokens (e.g., OpenAI *turbo* or local llama.cpp) or even rule-based fallbacks for the short-term agent.
- **Hardware footprint.** Storing months of tick-level data for dozens of symbols, plus embedding vectors for semantic memory, can push past a single GPU's vRAM. A pragmatic split is: indicators and memories on CPU-RAM, small LLM inference on GPU, long-term archive in S3 or in BigQuery.

Where to push from here

- **Richer semantic memory.** Pipe in live RSS headlines, Reddit sentiment, and macro calendars so the Macro agent's news search is genuinely real-time.
- **Adaptive personas.** Reinforcement-learn the agents' risk tolerance (or even their prompts) so they evolve with market feedback instead of staying static.
- **Cross-asset portfolio.** Position-size across equities, futures, and FX using a Kelly-scaled optimisation layer; correlation clustering can stop the agents from tripling down on the same factor bet.
- **Execution microservice.** A thin Go / Rust layer that listens to Debate decisions and slices orders via TWAP/VWAP would turn the research stack into a real trading stack.

Conclusion

Medium

[Home](#)[Library](#)[Profile](#)[Stories](#)[Stats](#)[Following](#)[Kiran Maan](#)[ODSC - Open Data ...](#)[Data Engineer Thin...](#)[Data Hackers](#)[tecnologia no Grup...](#)[DataDrivenInvestor](#)[Practicus AI](#)[The Startup](#)[Entrepreneurship ...](#)[Data Science Colle...](#)

Layered-Memory Trader shows that cognitively-inspired design — specialized agents, scoped memories, and a confidence-weighted debate—can do more than wiggle lines on a chart. In back-tests, it preserved capital in drawdowns, redeployed into momentum, and kept risk metrics in check. Is it production-ready? Absolutely not without better data, tighter latency, and serious monitoring, but the blueprint is there. As markets grow noisier and data-rich, systems that can reason about trades, not just compute indicators, will own the next edge.

Reference & further reading

To provide deeper context for the concepts discussed in this article, here are key references that readers can explore:

1. Li, Y. et al. “TradingGPT: Multi-Agent System with Layered Memory and Distinct Characters for Enhanced Financial Trading Performance.” arXiv preprint, 2023.

<https://arxiv.org/abs/2309.03736>

2. *Agent-based model*. In *ScienceDirect Topics*.

[Open in app ↗](#)



3. AmirZainol. (2024, November 10). *MACD + RSI + BB (Bollinger Bands) Indicator* [TradingView script]. TradingView.

<https://www.tradingview.com/script/PFeFMXKI-MACD-RSI-BB/>

4. PwC.de. (n.d.). *Data management — the cornerstone of digital finance transformation*. Retrieved July 23, 2025, from <https://www.pwc.de/en/finance-transformation/data-management.html>

Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data ...
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

Like what you read?

- ★ Star the repo:
<https://github.com/gael55x/LayeredMemoryTrader>
- 🔗 Connect on LinkedIn: [Gaille Amolong](#)
- 👏 Share this post (or give this post 50 claps)

. . .

A Message from InsiderFinance



Thanks for being a part of our community! Before you go:

- 👏 Clap for the story and follow the author 🙌
- 📰 View more content in the [InsiderFinance Wire](#)
- 📺 Take our [FREE Masterclass](#)
- 📊 Discover [Powerful Trading Tools](#)

Machine Learning

Finance

Ai Agent

AI

Algorithmic Trading



Follow

Published in InsiderFinance Wire

13.6K followers · Last published 1 day ago

InsiderFinance news wire — Top articles on data-driven trading, investing, analyses, tools, and strategies to achieve financial freedom



Follow

Written by Gaille Amolong

332 followers · 8 following

Software engineer. I build AI, study trading, write about productivity, tech, and life lessons. (I need a part-time job lol)
<https://gailleamolong.vercel.app/>

Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data ...
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

Responses (4)



Gabriel Pehls

What are your thoughts?



Mateusz Banaszak

Aug 8



So now we've gone from "let's read the tea leaves" to "let's have three different tea cups argue with each other before deciding what the leaves mean." Wrapping old technical indicators in a "debate" doesn't magically make them predictive, it just... [more](#)



4

[Reply](#)



Fabio Balloni

Aug 5



"Any production roll-out needs a broker API wrapper that simulates all four costs during walk-forward test"

To be more precise:

Any advertisement of a trading strategy needs to show a meaningful backtest analysis over a long horizon (years!) including... [more](#)

 4

 1 reply

[Reply](#)

 Arse

4 days ago



Hey everyone!

I'm a dev with a passion for quantitative finance, and I've spent the last few months building a web tool I wanted to share with you all to get some feedback.

The core idea: Price charts tell you what happened, but not always why. I... [more](#)

 2

[Reply](#)

See all responses

More from Gaille Amolong and InsiderFinance Wire

```
21:19:52.603522] Training ML model on 89 samples...
ples: 62; Testing samples: 27
on Report:
precision    recall  f1-score   support

      1.00      1.00      1.00        27

           1.00      1.00      1.00        27
      1.00      1.00      1.00        27
      1.00      1.00      1.00        27

trix:
```



In InsiderFinance Wire by Gaille Amolong

Meet the 2x Leverage ML-Based Spread Trading Model: 85 Straight Wins and 0.12 BTC...

Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan
-  ODSC - Open Data ...
-  Data Engineer Thin...
-  Data Hackers
-  tecnologia no Grup...
-  DataDrivenInvestor
-  Practicus AI
-  The Startup
-  Entrepreneurship ...
-  Data Science Colle...

Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •

Leveraging Machine Learning, Leverage, and CoinGecko's Data for Consistent Crypto Profits

Feb 21  131  5



 In InsiderFinance Wire by Michael Schwarz

The storm is coming: Here's exactly what I'm doing with my portfolio right now

After 40 years on Wall Street, I'm seeing all the warning signs again

Aug 13  4K  107



 In InsiderFinance Wire by Itay V

I tried trading with Agentic AI, and it's mind blowing

Discover how agentic systems are transforming the retail-trading scene and how you can use it to keep or discover...

May 27  1.2K  24

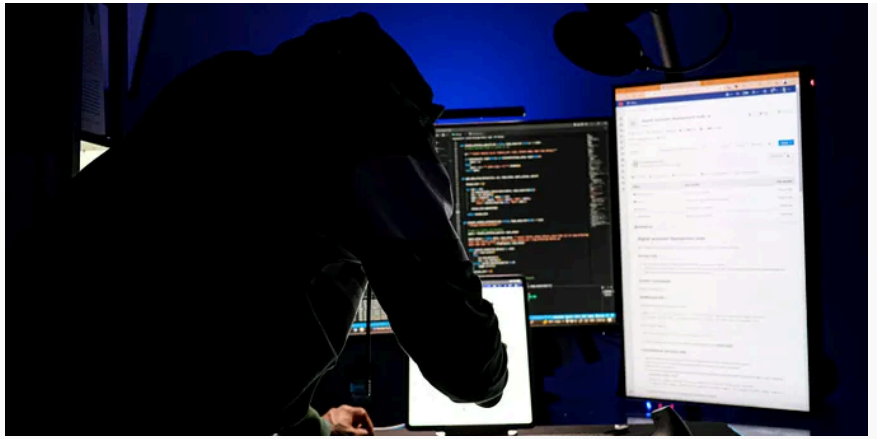


Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan •
-  ODSC - Open Data ... •
-  Data Engineer Thin... •
-  Data Hackers •
-  tecnologia no Grup... •
-  DataDrivenInvestor •
-  Practicus AI •
-  The Startup •
-  Entrepreneurship ... •
-  Data Science Colle... •



 Gaille Amolong

We were part of a Botnet: A GitLab Breach Post-Mortem

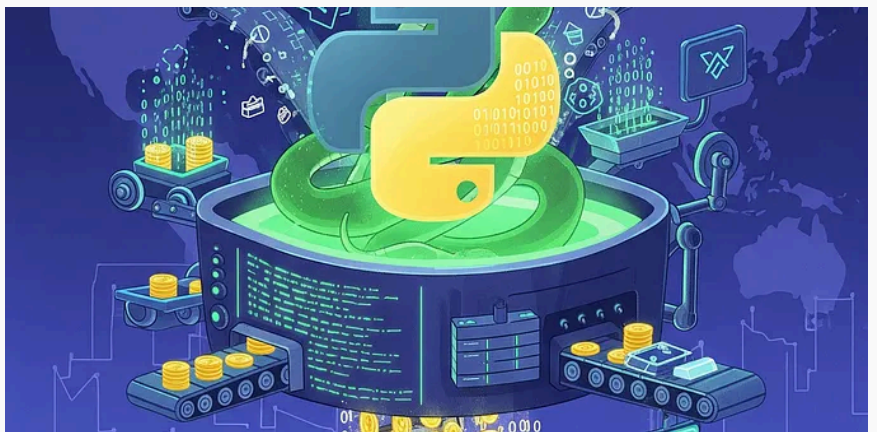
How one Saturday-night cryptojack forced us to rethink risk-management. Both in code and in cash.

★ Jul 30 🖱️ 103 💬 1  

See all from Gaille Amolong

See all from InsiderFinance Wire

Recommended from Medium



Medium

- Home
- Library
- Profile
- Stories
- Stats

Following

- Kiran Maan
- ODSC - Open Data ...
- Data Engineer Thin...
- Data Hackers
- tecnologia no Grup...
- DataDrivenInvestor
- Practicus AI
- The Startup
- Entrepreneurship ...
- Data Science Colle...

PY In Python in Plain English by Suleman Safdar

The Python Tool I Built in a Weekend That Now Pays My Rent

How I turned a tiny automation script into a paid product using libraries, clean OOP, and a little C++ speed where it...

★ Aug 11 🖱️ 2.5K 💬 43



IF In InsiderFinance Wire by Itay V

I tried trading with Agentic AI, and it's mind blowing

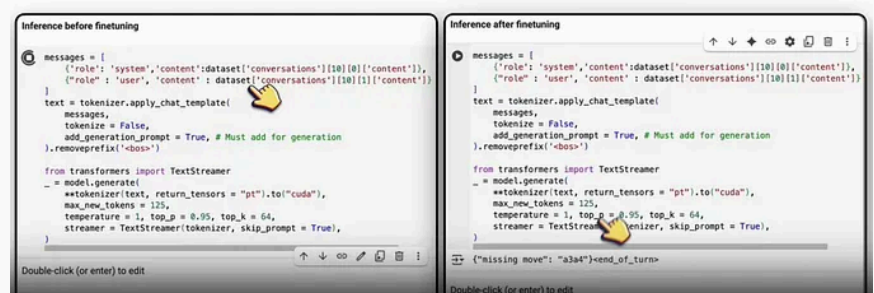
Discover how agentic systems are transforming the retail-trading scene and how you can use it to keep or discover...

May 27 🖱️ 1.2K 💬 24



Before Finetuning
(generates random chess moves)

After Finetuning
(finds the exact chess move)



CL In Coding Nexus by Civil Learning

Google's New LLM Runs on Just 0.5 GB RAM—Here's How to Fine-Tune It Locally

Medium

 Home Library Profile Stories Stats Following Kiran Maan • ODSC - Open Data Science • Data Engineer Thin... • Data Hackers • tecnologia no Grup... • DataDrivenInvestor • Practicus AI • The Startup • Entrepreneurship ... • Data Science Colle... •

A few days ago, Google quietly released a little AI model called Gemma 3 270M.

★ Aug 15 🖱️ 1.3K 💬 19

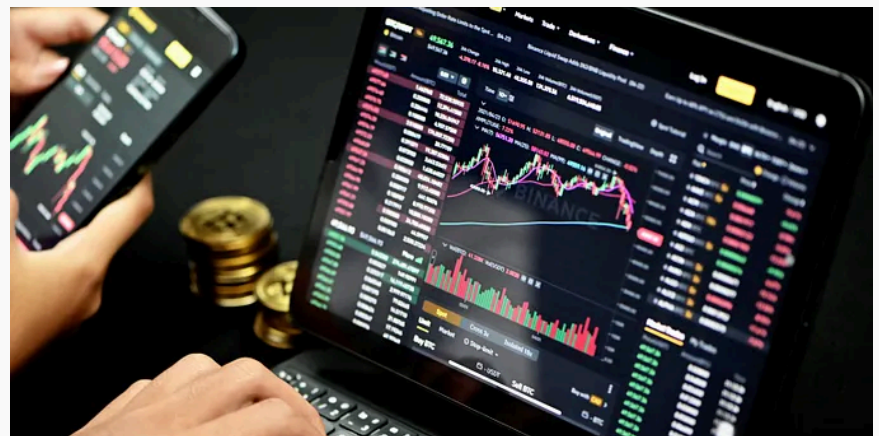


 In How To Profit AI by Mohamed Bakry

12 High-Selling Digital Products You Can Build with ChatGPT

Prompts are included with a Monetization Tool [Save it]

★ 1d ago 🖱️ 1.8K 💬 72



 In Mr. Plan ₿ Publication by Wealth Ways

The One Metric I Check Before Every Trade (It's Not RSI or MACD — And It Will Shock You!)

I Was Losing Money Until I Discovered This One Metric

★ Aug 5 🖱️ 64 💬 2



Medium

-  Home
-  Library
-  Profile
-  Stories
-  Stats

Following

-  Kiran Maan
-  ODSC - Open Data ...
-  Data Engineer Thin...
-  Data Hackers
-  tecnologia no Grup...
-  DataDrivenInvestor
-  Practicus AI
-  The Startup
-  Entrepreneurship ...
-  Data Science Colle...



 David Borst

Detecting Market Regimes: Hidden Markov Model

Detecting Market Regimes with Hidden Markov Models

4d ago

 89

 2





See more recommendations